

Team 7 CV-14 Guide

Ubuntu WSL2 Setup + Detection Message and Cross-Team Interface Alignment

This guide is intended as a practical reference for setting up the Team 7 ROS 2/Gazebo environment in Ubuntu WSL2 and understanding the core technical work for **CV-14 — Detection Message & Cross-Team Interface Alignment**.

The project bundle used for this guide contains:

- ROS 2 Jazzy + Gazebo simulation support;
- the Team 7 perception workspace;
- the YOLO11n-seg ONNX detector;
- custom `team7_msgs` detection messages;
- simulation-side interface documentation;
- camera and coordinate-frame documentation.

1. What CV-14 is actually about

CV-14 is primarily an **interface-contract task**, not an inference-model task.

Its purpose is to make sure the CV pipeline and downstream simulation/planning code agree on:

1. the ROS 2 output topic;
2. the ROS 2 message type and field layout;
3. the order of the four oriented-box corners;
4. the centroid definition;
5. class-name and confidence conventions;
6. pixel-space versus world-space coordinates;
7. the coordinate frame used for world positions;
8. behaviour for zero and multiple detections.

The key deliverable is a detection interface that both sides can implement without guessing.

2. Ubuntu WSL2 setup

2.1 Windows prerequisites

Use Windows 11 with WSL2 enabled. The supplied setup is designed for **Ubuntu 24.04 + ROS 2 Jazzy**.

If using an NVIDIA GPU, keep the NVIDIA driver on the Windows side. The supplied WSL guide explicitly says **not to install a separate Linux NVIDIA display driver inside WSL2**.

2.2 Install Ubuntu 24.04 under WSL2

Open **PowerShell as Administrator** and run:

```
[powershell]
wsl --install -d Ubuntu-24.04
```

Restart Windows if requested. Launch Ubuntu and create the Linux username/password.

If the WSL distribution itself has been moved to another Windows drive, that is fine. Inside Ubuntu, still work in the Linux filesystem such as `~/team7_project_bundle`, rather than running ROS/Gazebo directly from `/mnt/c/...` or `/mnt/d/...`.

2.3 Install ROS 2 Jazzy

Inside Ubuntu:

```
[bash]
sudo apt update && sudo apt install -y curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
sudo apt install -y ros-jazzy-desktop
```

Automatically source ROS 2 in new terminals:

```
[bash]
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Basic verification:

```
[bash]
ros2 --help
```

2.4 Copy the project into the WSL Linux filesystem

Do not run the project directly from the Windows-mounted path if possible. Copy it into the Ubuntu home directory.

Example:

```
[bash]
cp -r "/mnt/d/OnTrack Tasks/Ubuntu, WSL, Gazebo/team7_project_bundle" ~/
cd ~/team7_project_bundle
```

Change the Windows path to wherever the extracted bundle is located.

Make the supplied scripts executable:

```
[bash]
chmod +x run_team7_sim.sh run_team7_perception.sh \
team7_sim_mark2_v1.0/scripts/*.sh \
team7_sim_mark2_v1.0/overlay/setup_overlay.sh
```

2.5 Install simulation dependencies

```
[bash]
cd ~/team7_project_bundle/team7_sim_mark2_v1.0
bash scripts/install_dependencies.sh
```

The script installs the ROS/Gazebo/MoveIt packages required by the supplied simulation.

To check missing packages without installing:

```
[bash]
bash scripts/install_dependencies.sh --check
```

2.6 Start the Team 7 simulation

```
[bash]
bash ~/team7_project_bundle/run_team7_sim.sh
```

A successful first start should eventually report:

```
[text]
20 passed, 0 failed
Simulation is UP and verified.
```

The normal graphical outputs are:

- Gazebo Sim with the e-waste scene and Kinova arm;
- Team 7 camera viewer containing the overhead and side feeds.

2.7 Verify the overhead camera

Useful commands in another Ubuntu terminal:

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep camera
ros2 topic info /top_camera/image
ros2 topic info /top_camera/camera_info
ros2 topic echo /top_camera/image --once
```

The current overhead-camera SDF in the supplied bundle is configured as:

- topic: /top_camera/image;
- camera info: /top_camera/camera_info;
- resolution: **1280 × 960**;
- image format in Gazebo: R8G8B8;
- configured update rate: 2 Hz.

The ONNX detector still resizes/letterboxes the image to **640 × 640** internally, then maps results back to original-image coordinates.

Important documentation mismatch

Some older files in the bundle still mention **320 × 240**. In particular, the comment inside `PhoneDetection.msg` still says the original top-camera resolution is 320 × 240. That comment is stale relative to the updated `overhead_camera/model.sdf`, which uses 1280 × 960.

CV-14 should therefore define pixel fields as being in the **original source-image coordinates**, without hard-coding a resolution into the message contract.

2.8 Build the perception workspace

Install ONNX Runtime if necessary:

```
[bash]
sudo apt install -y python3-pip
python3 -m pip install --user onnxruntime
```

Optional GPU package:

```
[bash]
python3 -m pip install --user onnxruntime-gpu
```

Build:

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
colcon build --symlink-install
source install/setup.bash
```

2.9 Run the perception node

Leave Gazebo running and start the detector in another terminal:

```
[bash]
bash ~/team7_project_bundle/run_team7_perception.sh
```

Current CV output:

```
[bash]
ros2 topic echo /phone_detections
```

Useful checks:

```
[bash]
ros2 topic type /phone_detections
ros2 topic info /phone_detections -v
ros2 topic hz /phone_detections
ros2 interface show team7_msgs/msg/PhoneDetection
ros2 interface show team7_msgs/msg/PhoneDetectionArray
```

2.10 Stop the simulation cleanly

Use the supplied stop script instead of relying only on Ctrl+C:

```
[bash]
bash ~/team7_project_bundle/team7_sim_mark2_v1.0/scripts/stop_sim.sh
```

3. Current CV-side detection interface

The current perception node publishes:

```
[text]
Topic: /phone_detections
Type: team7_msgs/msg/PhoneDetectionArray
QoS: publisher depth 10
```

PhoneDetectionArray.msg currently contains:

```
[text]
std_msgs/Header header
PhoneDetection[] detections
```

Each PhoneDetection.msg currently contains:

```
[text]
std_msgs/Header header
string class_id
float64 score

int32 bbox_x1
int32 bbox_y1
int32 bbox_x2
int32 bbox_y2

float64 mar_center_x
float64 mar_center_y
float64 mar_width
float64 mar_height
float64 angle_deg

float64 world_x
float64 world_y
float64 world_z
bool world_valid
```

The current node publishes one PhoneDetectionArray for every processed image frame. Therefore, if inference returns no detections, the array is still published with:

```
[text]
detections: []
```

This is good interface behaviour because downstream nodes can distinguish **"detector is alive but sees nothing"** from **"detector is not publishing"**.

4. Current geometry conventions already implemented in the CV code

4.1 Axis-aligned bounding box

The fields:

```
[text]
bbox_x1, bbox_y1, bbox_x2, bbox_y2
```

are mapped back into the **original camera image**, not left in the 640×640 model-input space.

4.2 Oriented minimum-area rectangle

The segmentation mask is converted to an oriented minimum-area rectangle using OpenCV `minAreaRect`.

The current code computes:

- oriented-box centroid: `mar_center_x`, `mar_center_y`;
- width and height;
- angle;
- four actual corner points internally.

However, the current ROS message does **not yet publish those four corner points explicitly**.

That is an important CV-14/CV-15 boundary: CV-14 should freeze the corner convention; CV-15 can then publish the agreed four points.

4.3 Four-corner ordering already used internally

The function `order_mar_corners()` already imposes a deterministic order.

Its practical convention is:

1. start from the top-most corner in image coordinates;
2. if two candidates have the same vertical position, use the left-most one;
3. continue **clockwise in image coordinates**.

For a non-rotated rectangle this gives approximately:

```
[text]
corner 0 = top-left
corner 1 = top-right
corner 2 = bottom-right
corner 3 = bottom-left
```

For a rotated rectangle, the more precise wording should be used:

corner_0 is the point with smallest image y (tie: smallest x), and the remaining points proceed clockwise around the rectangle.

This convention is preferable to simply calling the corners TL/TR/BR/BL because a highly rotated rectangle may not have visually obvious "top-left" semantics.

4.4 Centroid convention

The current detector uses the centre of the segmentation-mask minimum-area rectangle:

```
[text]
centroid = (mar_center_x, mar_center_y)
```

This is more consistent with the oriented-box geometry than using the centre of the axis-aligned bounding box.

For CV-14, this should be explicitly documented so downstream consumers do not independently calculate a different centre.

5. The main CV-14 issue: the CV side and simulation side currently disagree

The supplied bundle contains **two different interface concepts**.

5.1 Current CV implementation

The current real detector uses:

```
[text]
/phone_detections
team7_msgs/msg/PhoneDetectionArray
```

It carries both pixel geometry and projected world coordinates.

5.2 Existing simulation-side frozen spec

sim/docs/DETECTION_INTERFACE.md defines an older/frozen simulation-side contract:

```
[text]
Topic: /detected_objects
```

```
Type: vision_msgs/msg/Detection3DArray
Frame: world
Units: metres
Rate: 2 Hz nominal
QoS: reliable, depth 10
```

It also expects class strings such as:

```
[text]
battery_cylindrical
battery_pouch
phone
phone_cover
```

The current ONNX model code, however, has only:

```
[python]
CLASS_NAMES = {0: "battery"}
```

So CV-14 must not pretend the interface is already aligned. The mismatch is exactly the work that CV-14 is meant to resolve.

6. CV-14 decisions that should be frozen with simulation/planning

Before marking CV-14 complete, explicitly agree on the following.

Decision 1 — final topic

Current CV topic:

```
[text]
/phone_detections
```

Older simulation contract:

```
[text]
/detected_objects
```

Choose one canonical cross-team topic, or formally define an adapter between them.

Decision 2 — final ROS 2 message type

Current CV type:

```
[text]
team7_msgs/msg/PhoneDetectionArray
```

Older simulation type:

```
[text]
vision_msgs/msg/Detection3DArray
```

The custom message is useful for CV because it can carry image-space geometry such as corners and centroids. Detection3DArray is useful for downstream world-space robotics.

A clean architecture is:

```
[text]
camera image
  ↓
CV detector
  ↓
custom CV detection message containing pixel geometry
  ↓
localisation / adapter
  ↓
world-space planning interface
```

That architecture should only be treated as final after the relevant simulation/planning members agree.

Decision 3 — four-corner order

Recommended wording based on the code already implemented:

```
[text]
corner_0: smallest image y, tie smallest x
corner_1: next clockwise
corner_2: next clockwise
corner_3: next clockwise
```

All corners are in original-image pixel coordinates.

Decision 4 — centroid

Recommended convention based on the existing segmentation post-processing:

```
[text]
centroid_x = mar_center_x
centroid_y = mar_center_y
```

That is the centre returned by `cv2.minAreaRect()` for the largest segmentation contour.

Decision 5 — class names

Do not leave model labels and interface labels ambiguous.

Current model output:

```
[text]
battery
```

Simulation's older class vocabulary includes:

```
[text]
battery_pouch
```

If the model's battery class specifically means the flat phone battery, a mapping such as:

```
[text]
battery -> battery_pouch
```

is plausible, but this should be **explicitly agreed**, not silently assumed.

Decision 6 — confidence

Use a floating-point confidence value in:

```
[text]
0.0 <= score <= 1.0
```

The current custom message already does this.

Decision 7 — pixel frame

Pixel-space fields should be documented as:

```
[text]
origin: top-left of original camera image
+x: right
+y: down
units: pixels
```

The phrase **original camera image** is important because inference internally uses a letterboxed 640 × 640 tensor.

Decision 8 — world frame

The simulation documentation expects world-space positions in:

```
[text]
frame: world
units: metres
```

The current custom message has explicit fields named `world_x`, `world_y`, and `world_z`, plus `world_valid`.

However, the message header is copied from the input camera image. The current camera header uses a Gazebo scoped sensor name such as:

```
[text]
overhead_camera::mount::overhead_camera
```

That header is **not the same thing as saying the `world_x/y/z` fields are in `world`**.

Therefore the cross-team contract must clearly state:

- the header refers to the source image/timestamp;
- pixel fields are in that image;
- `world_x/y/z`, when `world_valid=true`, are explicitly in Gazebo's world coordinate frame and use metres.

Longer-term, a `PoseStamped` or another explicitly framed world-position field would make this less ambiguous.

Decision 9 — zero detections

Publish an empty collection every processed frame:

```
[text]
detections: []
```

Do not simply stop publishing when nothing is detected.

Decision 10 — multiple detections

Publish all detections from the same image in one array message. Do not require consumers to assume detection index 0 is the best object unless the contract explicitly says so.

7. Suggested CV-14 message update before CV-15

The current message stores MAR centre/size/angle but not explicit four-corner coordinates. Since CV-15 requires the four ordered corners to be published, CV-14 should freeze the fields before CV-15 implementation.

One simple extension is:

```
[text]
# oriented box, original-image pixel coordinates
float64 corner_0_x
float64 corner_0_y
float64 corner_1_x
float64 corner_1_y
float64 corner_2_x
float64 corner_2_y
float64 corner_3_x
float64 corner_3_y

# agreed centroid
float64 centroid_x
float64 centroid_y
```

The existing `mar_center_x/y` can either remain for compatibility or be renamed only if all consumers are updated together.

Do not make a breaking rename in one package without updating the publisher and every subscriber in the same integration change.

8. Practical CV-14 implementation workflow

Step 1 — bring up the stack

Terminal 1:

```
[bash]
bash ~/team7_project_bundle/run_team7_sim.sh
```

Terminal 2:

```
[bash]
source /opt/ros/jazzy/setup.bash
cd ~/team7_project_bundle/team7_perception_ws
source install/setup.bash
bash ~/team7_project_bundle/run_team7_perception.sh
```

Terminal 3:

```
[bash]
source /opt/ros/jazzy/setup.bash
cd ~/team7_project_bundle/team7_perception_ws
source install/setup.bash
ros2 topic echo /phone_detections
```

Step 2 — inspect the live interface

```
[bash]
ros2 topic type /phone_detections
ros2 topic info /phone_detections -v
ros2 interface show team7_msgs/msg/PhoneDetection
ros2 interface show team7_msgs/msg/PhoneDetectionArray
```

Keep screenshots or terminal evidence showing these outputs.

Step 3 — inspect the camera contract

```
[bash]
ros2 topic echo /top_camera/image --once
ros2 topic echo /top_camera/camera_info --once
```

Record at least:

- width/height;
- encoding;
- image frame_id;
- timestamp;
- camera intrinsic matrix K.

Step 4 — write/freeze the interface table

The agreed table should contain at least:

Item	Final convention
Detection topic	agreed topic
Message type	agreed ROS 2 type
Source image	`/top_camera/image`

Pixel coordinates	original-image coordinates
Pixel origin	top-left
Pixel axes	+x right, +y down
Corner order	smallest-y start, clockwise
Centroid	MAR/segmentation centroid
Class field	agreed exact class string vocabulary
Confidence	0.0–1.0
World frame	`world`
World units	metres
Zero detections	publish empty array
Multiple detections	all detections in one frame-level array

Step 5 — update message definitions only after agreement

If PhoneDetection.msg changes:

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
colcon build --symlink-install
source install/setup.bash
```

If ROS interface generation behaves strangely after a message change, use a clean rebuild:

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
rm -rf build install log
colcon build --symlink-install
source install/setup.bash
```

Step 6 — verify the publisher still works

```
[bash]
ros2 topic echo /phone_detections
ros2 topic hz /phone_detections
```

Check:

- one array per processed frame;
- class and confidence populated;
- pixel geometry in valid image bounds;
- corner order deterministic;
- centroid visually matches the object;
- empty array is published when no object is found;
- all detections are included when multiple objects are present.

9. What evidence is worth keeping for CV-14

The strongest evidence set would be:

1. ROS 2 interface definition

- `ros2 interface show team7_msgs/msg/PhoneDetection`
- `ros2 interface show team7_msgs/msg/PhoneDetectionArray`

2. Live topic evidence

- `/phone_detections` containing class, confidence, centroid/corner geometry and world fields.

3. Camera metadata

- `/top_camera/image` showing current resolution and frame header;
- `/top_camera/camera_info` showing intrinsics.

4. Cross-team interface table or written agreement

- final topic;
- message type;
- class vocabulary;
- four-corner order;
- centroid definition;
- coordinate frame and units.

5. Empty-detection behaviour

- an example message with detections: `[]` while the node remains active.

6. Multiple-detection behaviour, if available

- one array containing more than one detection.

7. Source-code evidence

- the message file;
- corner-order implementation;
- publisher construction;
- final class mapping.

10. CV-14 completion checklist

CV-14 is ready to call complete when all of the following are true:

- ☐ Ubuntu WSL2 + ROS 2 Jazzy environment starts cleanly.
- ☐ Gazebo simulation starts and camera topics are available.

- ☐ Perception workspace builds successfully.
- ☐ ONNX detector publishes a live ROS 2 detection array.
- ☐ The current 1280×960 source-image convention is documented.
- ☐ Stale 320×240 message comments are removed or corrected.
- ☐ Final detection topic is agreed.
- ☐ Final ROS 2 message type is agreed.
- ☐ Four-corner order is explicitly documented.
- ☐ Centroid convention is explicitly documented.
- ☐ Class vocabulary is explicitly agreed.
- ☐ Confidence range is defined.
- ☐ Pixel coordinate convention is defined.
- ☐ World coordinate frame and metres are defined.
- ☐ Empty detections are represented by an empty array.
- ☐ Multiple detections are represented in one frame-level collection.
- ☐ Simulation/planning members can consume the interface without interpreting undocumented assumptions.

11. Relationship to the next CV tasks

CV-14 freezes the contract. It should make later work straightforward:

```
[text]
CV-14  define and align the interface
      ↓
CV-15  publish ordered corners + centroid using that interface
      ↓
CV-16  publish annotated debug images
      ↓
CV-17  convert the agreed image centroid to world coordinates and validate it
```

This is why CV-14 is important even though it contains less algorithmic work than CV-17: if the interface is ambiguous, every downstream component can be individually "correct" and still fail integration.